

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Medium (LABORATORY PERFORMANCE)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO4: *Design and develop GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. (BL3,BL5)*

Assessment Tool Weightage: 10%

QUESTIONS: 1

Total Marks: 50

ANNEXURE A

LABORATORY PERFORMANCE

Code of Conduct:

I NEELAKANDAMOORTHY C(192421063)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: NEELAKANDAMOORTHY C(192421063)

ANNEXURE

LABORATORY PERFORMANCE

1. Student Registration Applet

Develop a Java Applet for a university student registration form. The applet should display labels and input controls for Student Name, Register Number, Department, Gender, and Email ID. The application should use appropriate **AWT controls** such as Label, TextField, Checkbox, CheckboxGroup, and Button. Use a suitable **layout manager** to organize the controls. When the student clicks the **Submit** button, the entered information should be displayed within the applet window.

The student must demonstrate:

1. Applet initialization and architecture.
2. AWT controls.
3. Layout management.
4. Button event handling.
5. Appropriate event listener interface.
6. Display of user-entered information.

StudentRegistration.java

```
import java.applet.Applet;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
public class StudentRegistration extends Applet implements ActionListener {
```

```
    Label nameLabel, regLabel, genderLabel, emailLabel, titleLabel;
```

```
    TextField nameField, regField, emailField;
```

```
    Checkbox male, female;
```

```
CheckboxGroup genderGroup;

Button submitButton;

TextArea result;

public void init() {

    setLayout(new BorderLayout(10, 10));

    setBackground(new Color(230, 240, 255));

    titleLabel = new Label("STUDENT REGISTRATION FORM", Label.CENTER);

    titleLabel.setFont(new Font("Arial", Font.BOLD, 22));

    titleLabel.setForeground(Color.WHITE);

    titleLabel.setBackground(new Color(40, 80, 150));

    add(titleLabel, BorderLayout.NORTH);

    Panel formPanel = new Panel(new GridLayout(5, 2, 10, 10));

    formPanel.setBackground(new Color(230, 240, 255));

    nameLabel = new Label("Student Name:");

    nameField = new TextField();

    regLabel = new Label("Register Number:");

    regField = new TextField();

    genderLabel = new Label("Gender:");

    genderGroup = new CheckboxGroup();

    male = new Checkbox("Male", genderGroup, false);
```

```
female = new Checkbox("Female", genderGroup, false);

emailLabel = new Label("Email ID:");

emailField = new TextField();

submitButton = new Button("SUBMIT");

submitButton.setBackground(new Color(40, 150, 80));

submitButton.setForeground(Color.WHITE);

submitButton.setFont(new Font("Arial", Font.BOLD, 14));

formPanel.add(nameLabel);

formPanel.add(nameField);

formPanel.add(regLabel);

formPanel.add(regField);

formPanel.add(genderLabel);

Panel genderPanel = new Panel();

genderPanel.setBackground(new Color(230, 240, 255));

genderPanel.add(male);

genderPanel.add(female);

formPanel.add(genderPanel);

formPanel.add(emailLabel);

formPanel.add(emailField);

formPanel.add(new Label(""));
```

```
formPanel.add(submitButton);

add(formPanel, BorderLayout.CENTER);

result = new TextArea();

result.setEditable(false);

result.setBackground(Color.WHITE);

result.setForeground(new Color(30, 60, 120));

result.setFont(new Font("Arial", Font.BOLD, 14));

Panel resultPanel = new Panel(new BorderLayout());

resultPanel.setBackground(new Color(210, 225, 245));

resultPanel.add(new Label("REGISTRATION DETAILS"), BorderLayout.NORTH);

resultPanel.add(result, BorderLayout.CENTER);

add(resultPanel, BorderLayout.SOUTH);

submitButton.addActionListener(this);

}

public void actionPerformed(ActionEvent e) {

    String name = nameField.getText();

    String reg = regField.getText();

    String email = emailField.getText();

    Checkbox selectedGender = genderGroup.getSelectedCheckbox();
```

```
        if (name.equals("") || reg.equals("") || email.equals("") || selectedGender == null) {

            result.setText("Please enter all details.");

            return;

        }

        result.setText(

            "Student Name    : " + name +

            "\nRegister Number : " + reg +

            "\nGender          : " + selectedGender.getLabel() +

            "\nEmail ID        : " + email

        );

    }

}
```

StudentRegistration.html

```
<html>

<body>

<applet code="StudentRegistration.class" width="700" height="550">

</applet>

</body>

</html>
```

OUTPUT:

Applet Viewer: StudentRegistration.class

Applet

STUDENT REGISTRATION FORM

Student Name	NEELAKANDAMOORTHY C
Register Number	192421063
Department	Information Technology
Gender	☒ Male ☐ Female
Email ID	192421063.SIMATS @SAVEETHA.COM

SUBMIT

Applet Viewer: StudentRegistration.class

Applet

STUDENT REGISTRATION FORM

Student Name	NEELAKANDAMOORTHY C
Register Number	192421063
Department	Information Technology
Gender	☒ Male ☐ Female
Email ID	192421063.SIMATS @SAVEETHA.COM
SUBMIT	

STUDENT DETAILS

Name : NEELAKANDAMOORTHY C
Register No: 192421063
Department : Information Technology
Gender : Male
Email : 192421063.SIMATS @SAVEETHA.COM

Applet started.

2. Simple Calculator Applet

Develop an Applet-based calculator that accepts two numbers and provides buttons for **Addition, Subtraction, Multiplication, and Division.**

The application should use TextField, Label, and Button controls. Appropriate event listeners must be implemented so that clicking each button performs the corresponding operation.

The program must handle invalid numeric input and division by zero appropriately.

The student must demonstrate:

1. Basic Applet structure.
2. AWT controls.
3. Event sources and event objects.
4. ActionListener.
5. Exception handling.
6. Display methods.

CalculatorApplet.java :

```
import java.applet.Applet;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
public class CalculatorApplet extends Applet implements ActionListener {
```

```
    TextField num1, num2, result;
```

```
    Button add, sub, mul, div;
```

```
    Label title;
```

```
    public void init() {
```

```
        setLayout(new BorderLayout(10, 10));
```

```
setBackground(new Color(235, 245, 255));

title = new Label("CALCULATOR APPLET", Label.CENTER);

title.setFont(new Font("Arial", Font.BOLD, 24));

title.setForeground(Color.WHITE);

title.setBackground(new Color(45, 90, 160));

add(title, BorderLayout.NORTH);

Panel inputPanel = new Panel(new GridLayout(3, 2, 10, 10));

inputPanel.setBackground(new Color(235, 245, 255));

inputPanel.add(new Label("First Number:"));

num1 = new TextField();

inputPanel.add(num1);

inputPanel.add(new Label("Second Number:"));

num2 = new TextField();

inputPanel.add(num2);

inputPanel.add(new Label("Result:"));

result = new TextField();

result.setEditable(false);

result.setBackground(Color.WHITE);

inputPanel.add(result);

add(inputPanel, BorderLayout.CENTER);
```

```
Panel buttonPanel = new Panel(new FlowLayout());
```

```
add = new Button("ADD");
```

```
sub = new Button("SUBTRACT");
```

```
mul = new Button("MULTIPLY");
```

```
div = new Button("DIVIDE");
```

```
add.setBackground(new Color(40, 160, 80));
```

```
sub.setBackground(new Color(220, 150, 40));
```

```
mul.setBackground(new Color(70, 110, 190));
```

```
div.setBackground(new Color(200, 60, 60));
```

```
add.setForeground(Color.WHITE);
```

```
sub.setForeground(Color.WHITE);
```

```
mul.setForeground(Color.WHITE);
```

```
div.setForeground(Color.WHITE);
```

```
buttonPanel.add(add);
```

```
buttonPanel.add(sub);
```

```
buttonPanel.add(mul);
```

```
buttonPanel.add(div);
```

```
add(buttonPanel, BorderLayout.SOUTH);
```

```
add.addActionListener(this);
```

```
sub.addActionListener(this);
```

```
mul.addActionListener(this);

div.addActionListener(this);

}

public void actionPerformed(ActionEvent e) {

    try {

        double a = Double.parseDouble(num1.getText());

        double b = Double.parseDouble(num2.getText());

        double answer = 0;

        if (e.getSource() == add) {

            answer = a + b;

        } else if (e.getSource() == sub) {

            answer = a - b;

        } else if (e.getSource() == mul) {

            answer = a * b;

        } else if (e.getSource() == div) {

            if (b == 0) {

                result.setText("Cannot divide by zero");

                return;

            }

            answer = a / b;

        }

    }

}
```

```
    }

    result.setText(String.valueOf(answer));

} catch (NumberFormatException ex) {

    result.setText("Enter valid numbers");

}

}

}
```

CalculatorApplet.html :

```
<html>

<body>

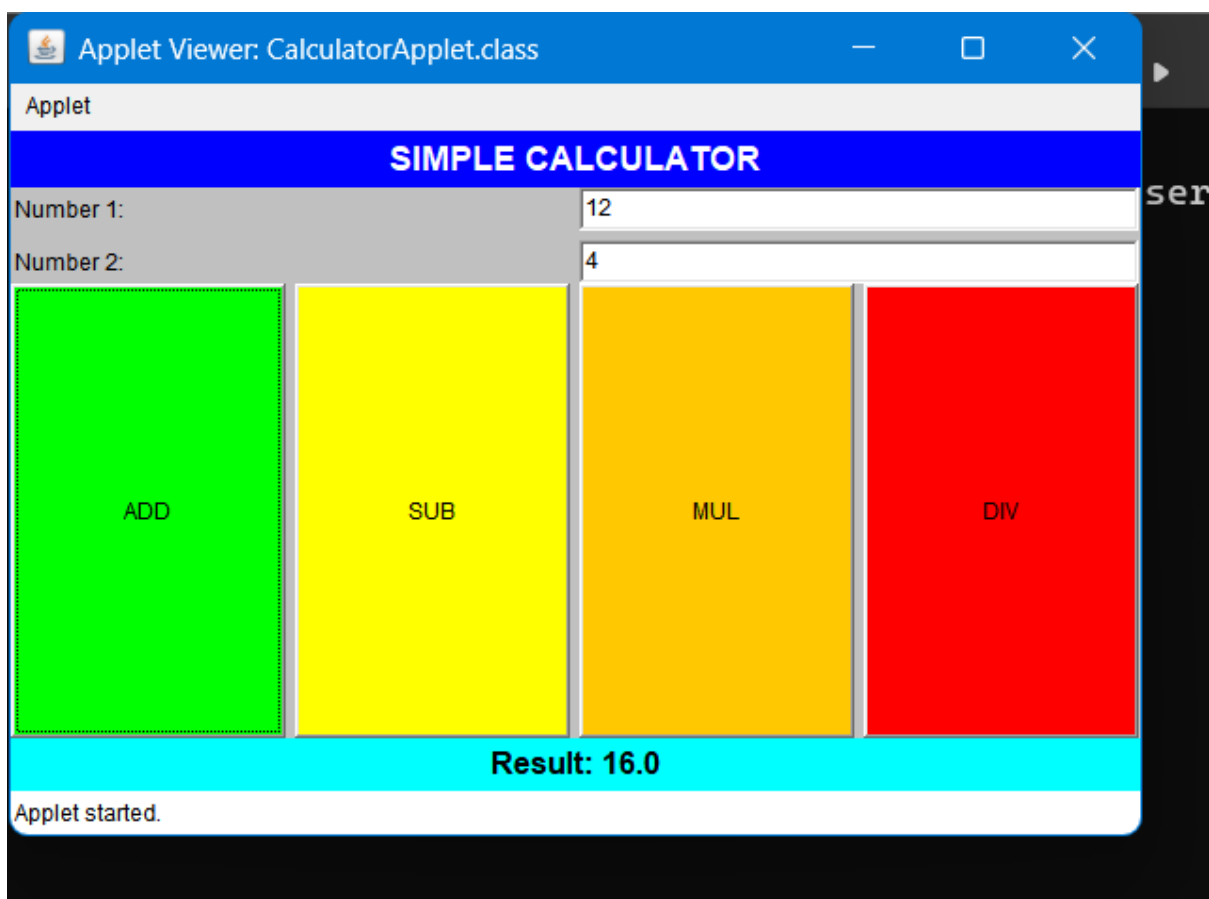
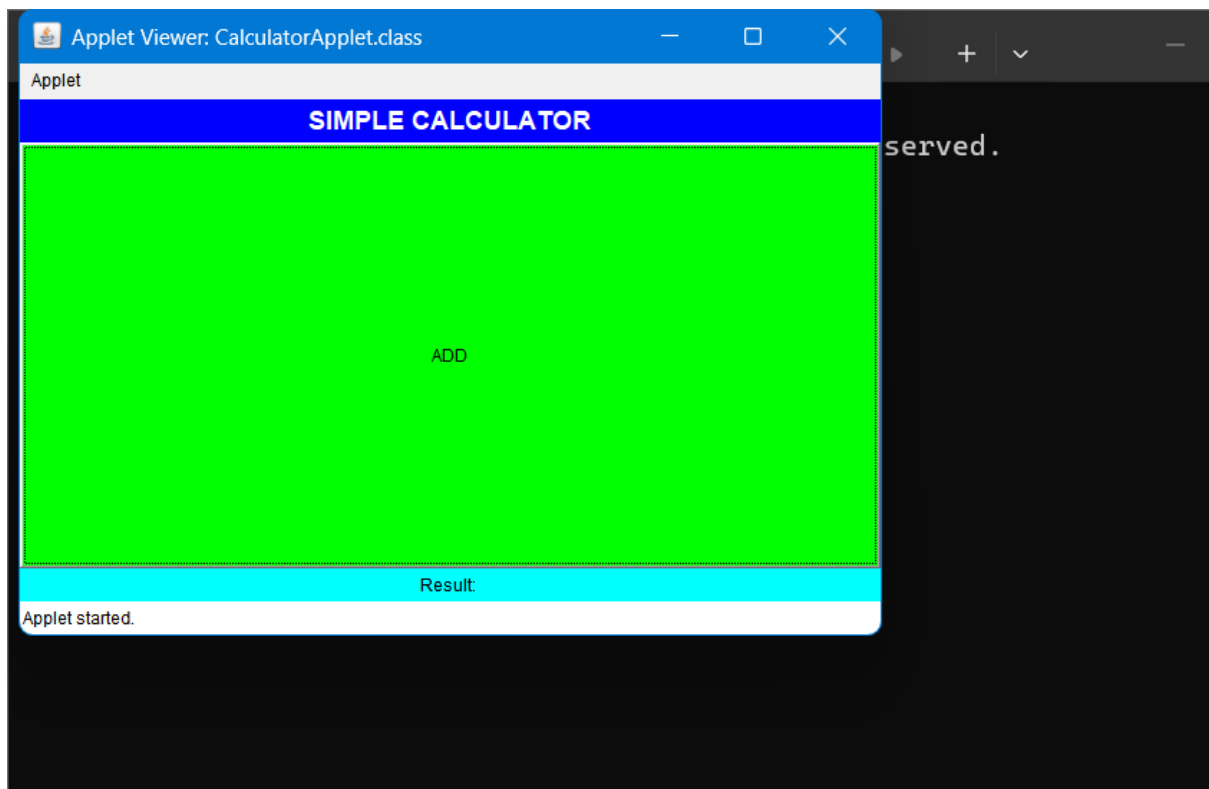
<applet code="CalculatorApplet.class" width="700" height="450">

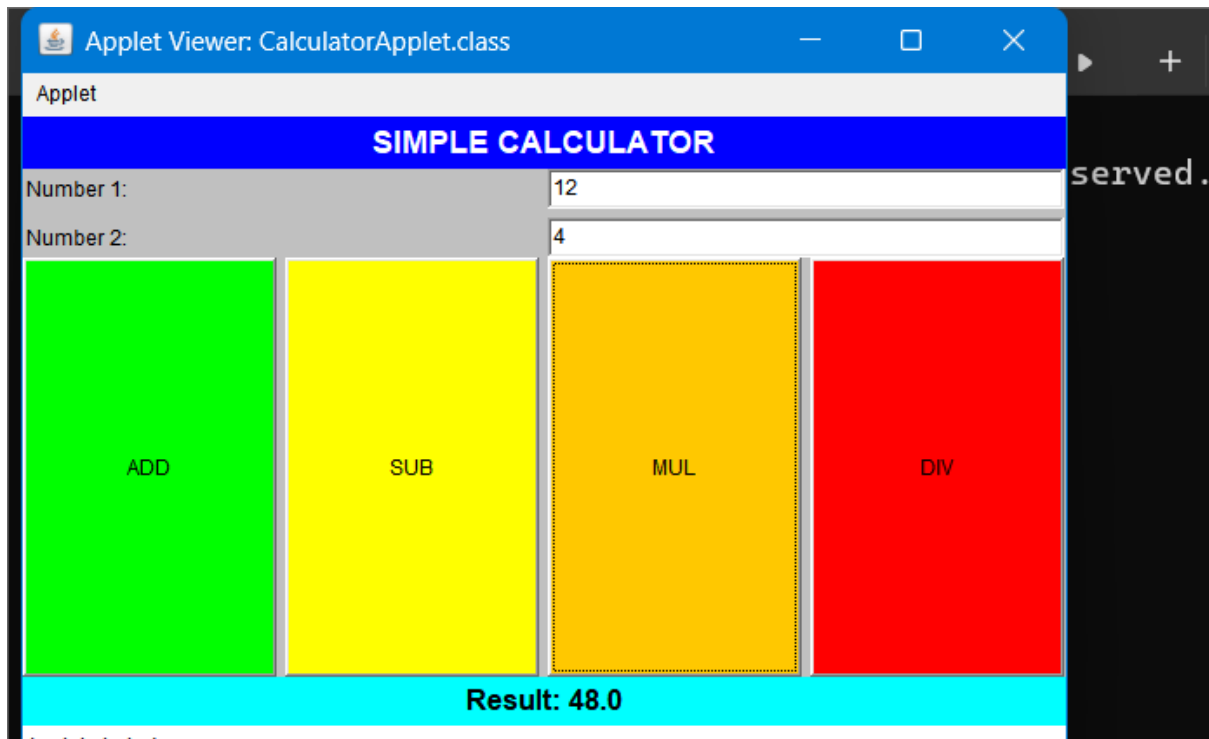
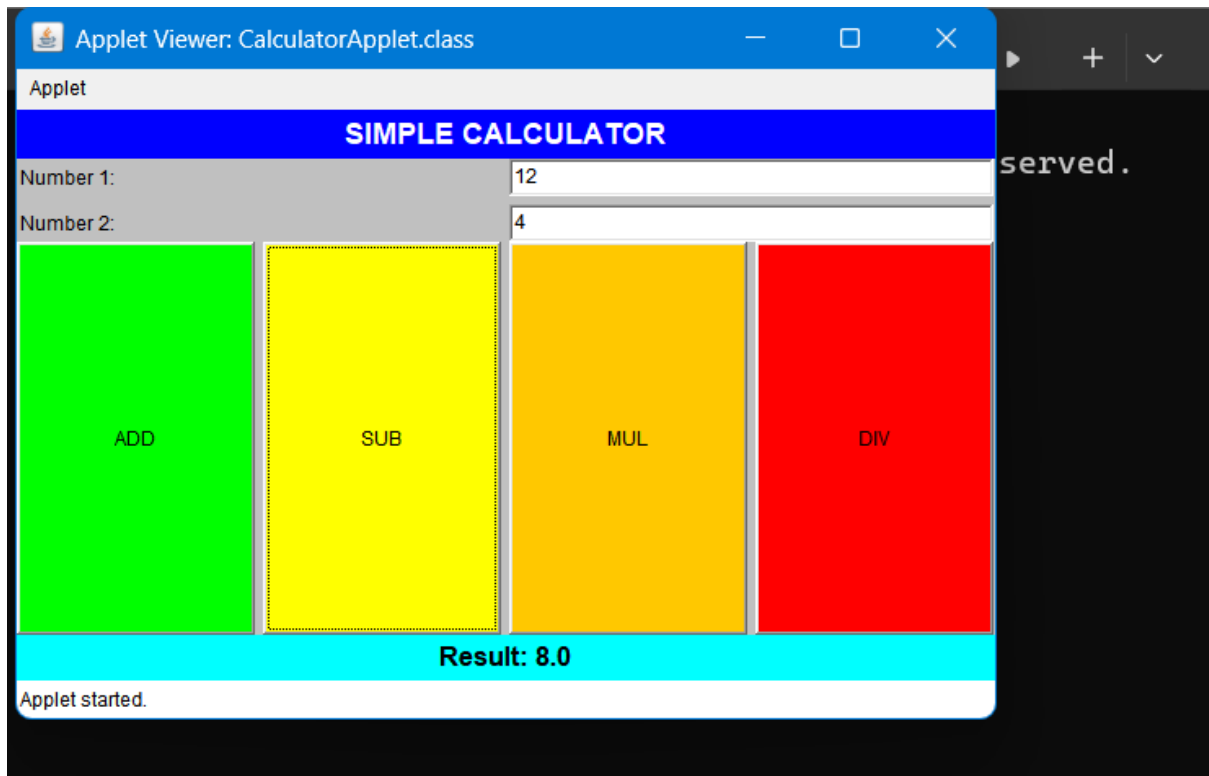
</applet>

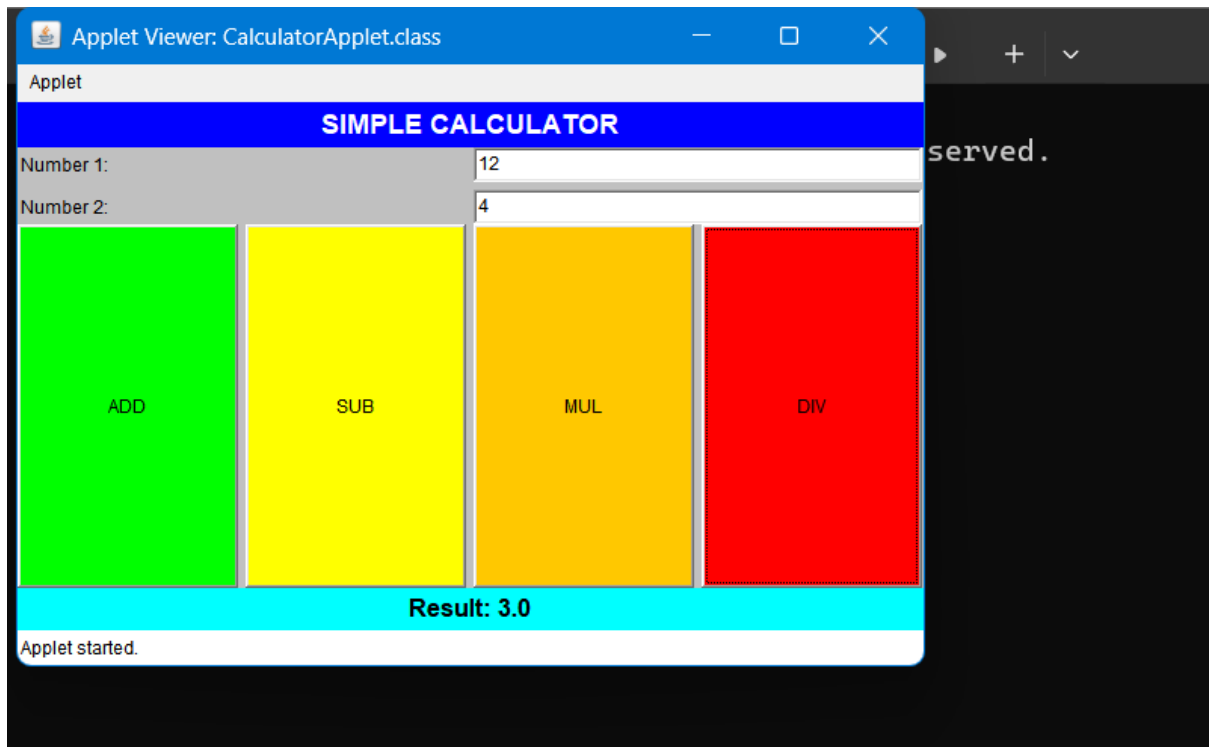
</body>

</html>
```

OUTPUT :







3. Course Selection Applet

Develop a university course selection Applet where students can select:

- Department
- Semester
- Course
- Elective type

Use Choice, List, Checkbox, and Button controls wherever appropriate.

When the student clicks **Register**, display the selected courses and other information.

Use appropriate layout managers and event listeners.

CourseSelection.java :

```
import java.applet.Applet;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
public class CourseSelection extends Applet implements ActionListener {
```

```
    Choice department, semester, course;
```

```
    Checkbox regular, elective;
```

```
    CheckboxGroup typeGroup;
```

```
    Button registerButton;
```

```
    TextArea result;
```

```
    Label title;
```

```
    public void init() {
```

```
        setLayout(new BorderLayout(10, 10));
```

```
        setBackground(new Color(235, 245, 255));
```

```
        title = new Label("UNIVERSITY COURSE SELECTION", Label.CENTER);
```

```
        title.setFont(new Font("Arial", Font.BOLD, 22));
```

```
        title.setForeground(Color.WHITE);
```

```
title.setBackground(new Color(45, 90, 160));

add(title, BorderLayout.NORTH);

Panel formPanel = new Panel(new GridLayout(5, 2, 10, 10));

formPanel.setBackground(new Color(235, 245, 255));

formPanel.add(new Label("Department:"));

department = new Choice();

department.add("Computer Science");

department.add("Information Technology");

department.add("Electronics");

department.add("Mechanical");

formPanel.add(department);

formPanel.add(new Label("Semester:"));

semester = new Choice();

semester.add("Semester 1");

semester.add("Semester 2");

semester.add("Semester 3");

semester.add("Semester 4");

semester.add("Semester 5");

semester.add("Semester 6");

formPanel.add(semester);
```

```
formPanel.add(new Label("Course:"));

course = new Choice();

course.add("Java Programming");

course.add("Data Structures");

course.add("Database Management");

course.add("Computer Networks");

course.add("Operating Systems");

formPanel.add(course);

formPanel.add(new Label("Elective Type:"));

typeGroup = new CheckboxGroup();

regular = new Checkbox("Regular", typeGroup, true);

elective = new Checkbox("Elective", typeGroup, false);

Panel typePanel = new Panel();

typePanel.add(regular);

typePanel.add(elective);

formPanel.add(typePanel);

registerButton = new Button("REGISTER");

registerButton.setBackground(new Color(40, 150, 80));

registerButton.setForeground(Color.WHITE);

registerButton.setFont(new Font("Arial", Font.BOLD, 14));
```

```

formPanel.add(new Label(""));

formPanel.add(registerButton);

add(formPanel, BorderLayout.CENTER);

result = new TextArea();

result.setEditable(false);

result.setBackground(Color.WHITE);

result.setForeground(new Color(30, 60, 120));

result.setFont(new Font("Arial", Font.BOLD, 14));

Panel resultPanel = new Panel(new BorderLayout());

resultPanel.setBackground(new Color(210, 225, 245));

resultPanel.add(new Label("SELECTION DETAILS"), BorderLayout.NORTH);

resultPanel.add(result, BorderLayout.CENTER);

add(resultPanel, BorderLayout.SOUTH);

registerButton.addActionListener(this);

}

public void actionPerformed(ActionEvent e) {

    Checkbox selectedType = typeGroup.getSelectedCheckbox();

    result.setText(

        "Department  : " + department.getSelectedItem() +

        "\nSemester   : " + semester.getSelectedItem() +

```

```
        "\nCourse      : " + course.getSelectedItem() +  
  
        "\nElective Type: " + selectedType.getLabel() +  
  
        "\n\nCourse registered successfully!"  
  
    );  
  
    }  
  
}
```

CourseSelection.html :

```
<html>  
  
<body>  
  
<applet code="CourseSelection.class" width="700" height="500">  
  
</applet>  
  
</body>  
  
</html>
```

OUTPUT :

Applet Viewer: CourseSelectionApplet.class

Applet

UNIVERSITY COURSE SELECTION

Department

Semester

Course
Web Technology
Database Management
Computer Networks

Elective Type

REGISTER

Applet Viewer: CourseSelectionApplet.class

Applet

UNIVERSITY COURSE SELECTION

Department: Information Technology

Semester: 1

Course: Java Programming
Web Technology
Database Management
Computer Networks

Elective Type: Core

REGISTER

COURSE REGISTRATION DETAILS

Department : Information Technology
Semester : 1
Course : Java Programming
Elective : Core

Applet started.

4. Traffic Signal Control Applet

Develop an AWT Applet that simulates a traffic signal with three options:

- Red
- Yellow
- Green

Use appropriate controls to allow the user to select a signal. When the user selects a signal, display the corresponding message such as **STOP**, **WAIT**, or **GO**.

Use appropriate event sources, event classes, and listener interfaces.

TrafficSignal.java :

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class TrafficSignal extends Applet implements ActionListener {
    Button redButton, yellowButton, greenButton;
    Label title, message;
    Panel signalPanel;

    public void init() {
        setLayout(new BorderLayout(10, 10));
        setBackground(new Color(230, 240, 250));
        title = new Label("TRAFFIC SIGNAL CONTROL", Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 24));
        title.setForeground(Color.WHITE);
        title.setBackground(new Color(50, 70, 100));
        add(title, BorderLayout.NORTH);
        signalPanel = new Panel();
        signalPanel.setLayout(new GridLayout(3, 1, 10, 10));
        signalPanel.setBackground(Color.BLACK);
        redButton = new Button("RED");
        yellowButton = new Button("YELLOW");
```

```

greenButton = new Button("GREEN");
redButton.setBackground(Color.RED);
redButton.setForeground(Color.WHITE);
yellowButton.setBackground(Color.YELLOW);
yellowButton.setForeground(Color.BLACK);
greenButton.setBackground(Color.GREEN);
greenButton.setForeground(Color.BLACK);
signalPanel.add(redButton);
signalPanel.add(yellowButton);
signalPanel.add(greenButton);
add(signalPanel, BorderLayout.CENTER);
message = new Label("Select a traffic signal", Label.CENTER);
message.setFont(new Font("Arial", Font.BOLD, 20));
message.setBackground(Color.WHITE);
add(message, BorderLayout.SOUTH);
redButton.addActionListener(this);
yellowButton.addActionListener(this);
greenButton.addActionListener(this);
}

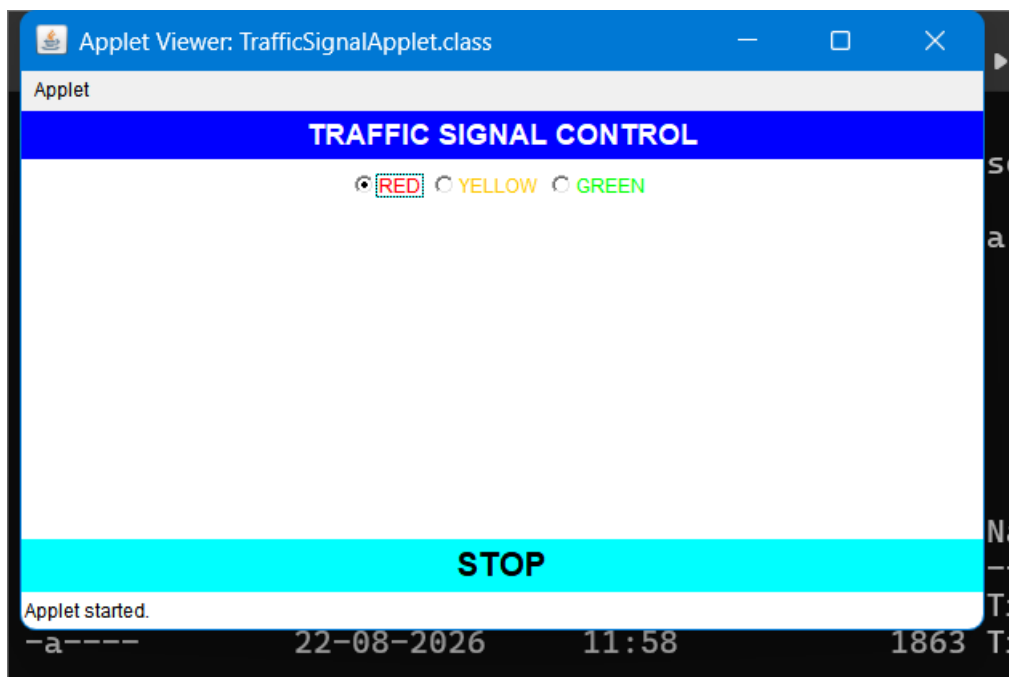
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == redButton) {
        message.setText("STOP");
        message.setForeground(Color.RED);
    } else if (e.getSource() == yellowButton) {
        message.setText("WAIT");
        message.setForeground(new Color(200, 150, 0));
    } else if (e.getSource() == greenButton) {
        message.setText("GO");
        message.setForeground(new Color(0, 150, 0));
    }
}
}

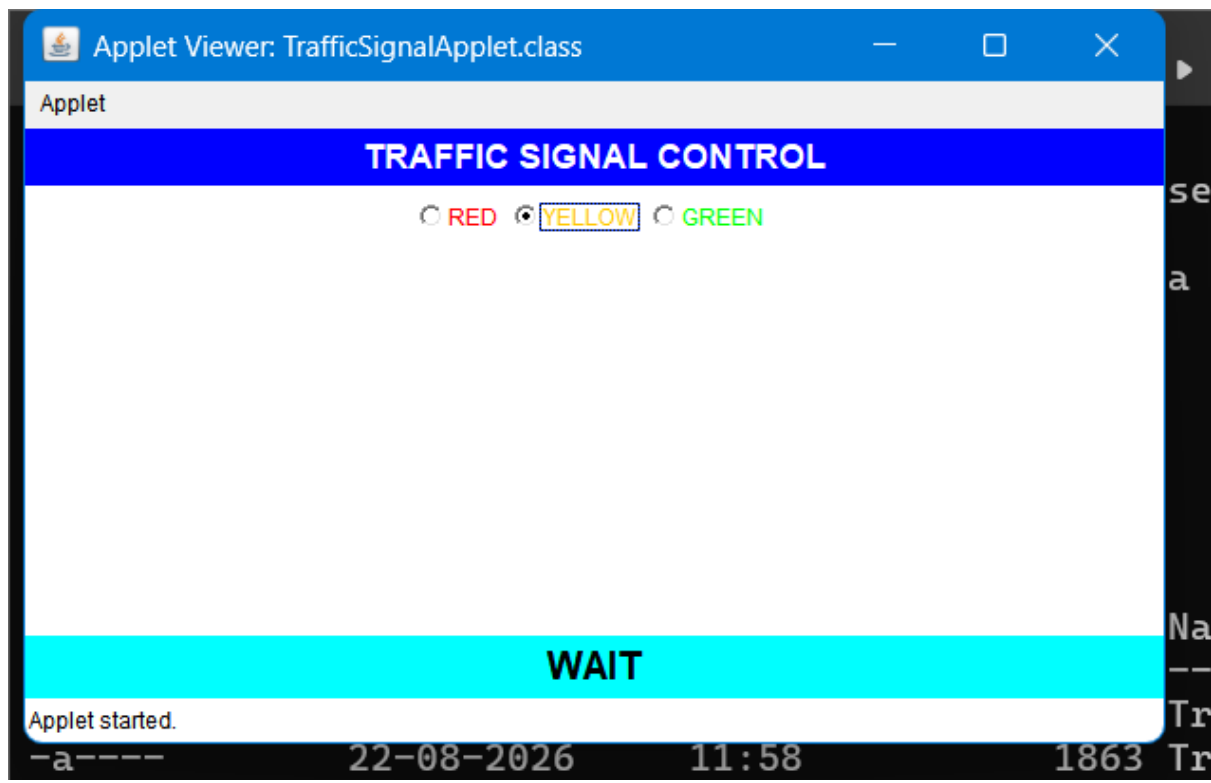
```

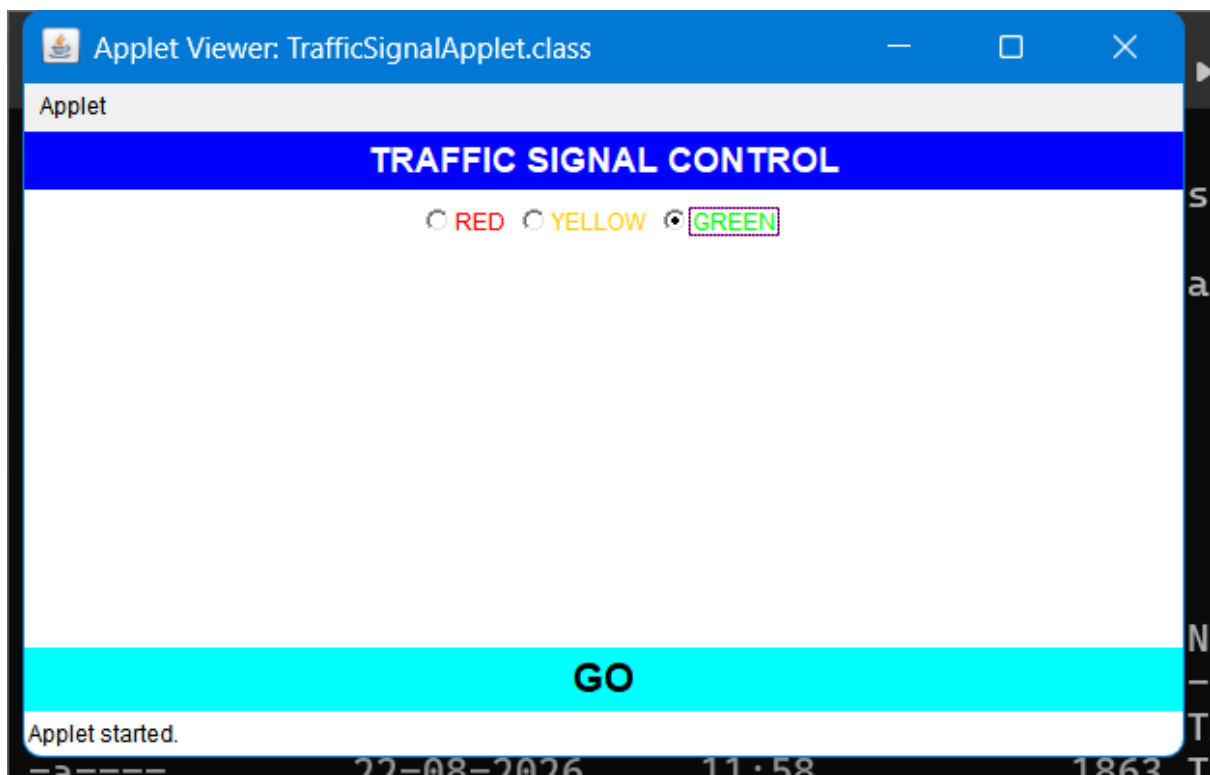
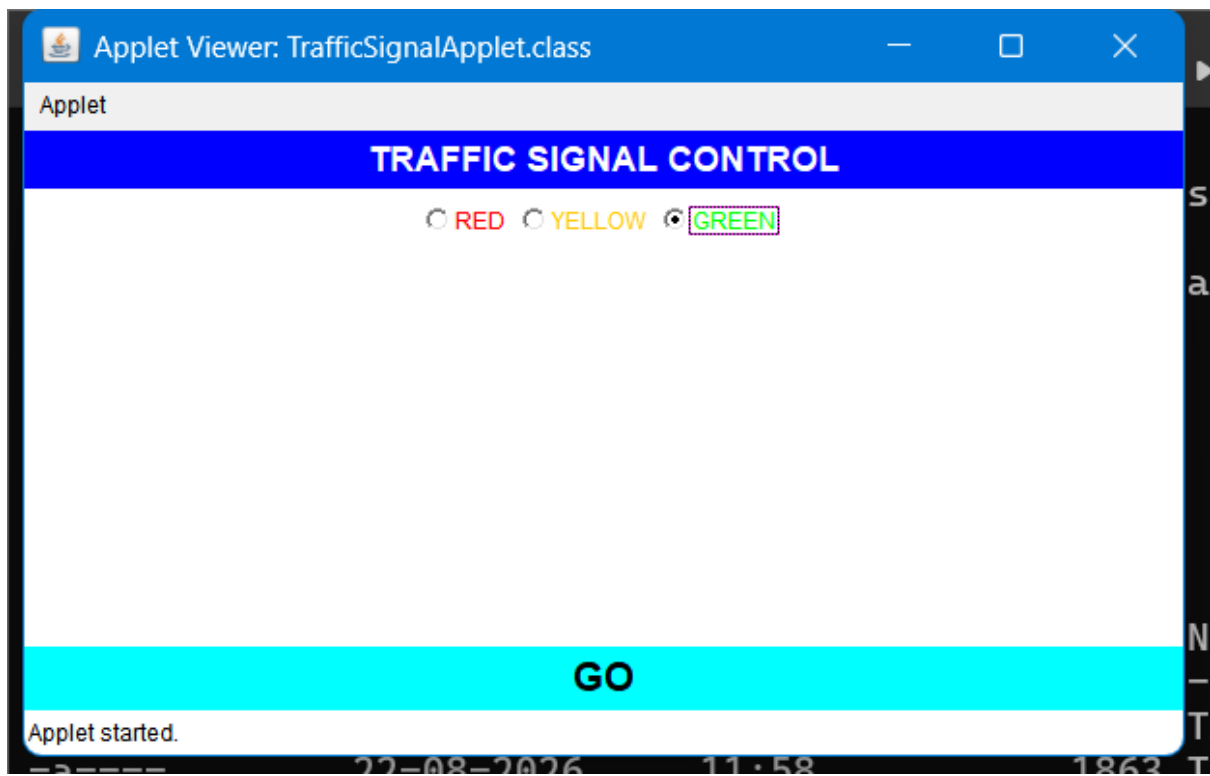
TrafficSignal.html :

```
<html>  
<body>  
<applet code="TrafficSignal.class" width="600" height="500">  
</applet>  
</body>  
</html>
```

OUTPUT :







RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Program Design	20%	Clearly understands the problem and develops an effective design	Good understanding and logical design	Basic understanding with some gaps	Poor understanding and unclear design
Java Implementation & Concept Application	30%	Applies Java concepts correctly and effectively	Applies concepts with minor errors	Limited application of concepts	Incorrect or incomplete implementation
Program Execution & Output	20%	Program executes correctly and produces accurate output	Executes with minor errors	Partially executes with some errors	Fails to execute correctly
Testing & Debugging	15%	Thoroughly tests and resolves errors independently	Tests and resolves most errors	Performs limited testing and debugging	Unable to test or debug effectively
Documentation & Presentation	15%	Complete, clear, and well-organized documentation	Good documentation with minor gaps	Basic documentation with some gaps	Incomplete or unclear documentation